



PROJECT PLAN

[Owl Eye Monitoring System]



FEBRUARY 7, 2025

PREPARED BY: TEAM 25-02

IAN RIGGINS, ISA SIDDIQUE, EVAN VALENCIA, HUY NGUYEN, WILLIAM WALKER

TABLE OF CONTENTS

Table of Contents.....	ii
Table of Contents Cont... ..	iii
1) Introduction.....	1
2) Project Overview	2
2.1) Scope	2
2.1.1) Identification.....	2
2.1.2) System Overview.....	2
2.1.3) Document Overview.....	2
2.2) Current System or Situation	2
2.3) Background, Objectives, and Scope.....	2
2.4) Operational Policies and Constraints	3
2.5) Description of Current System or Situation	3
2.6) Users or Involved Personnel	3
2.7) Standards to Follow	3
3) Development Background/Approach	5
3.1) High Level Estimates	5
3.1.1) Project Plan and Design Phase	5
3.1.2) Troublemaker Development Phase.....	5
3.1.3) Monitor/Alert System Development Phase	5
3.2) Key Contacts and Stakeholders.....	5
4) Features, Primary Deliverables, and External Commitments.....	7
4.1) Feature List.....	7
4.1.1) Troublemaker.....	7
4.1.2) Monitor and Alert System	7
4.2) Customer Deliverables	8
5) Project Schedule	10
5.1) Project Milestones	10
5.2) Project Status Tracking & Working Meeting Minutes.....	10
6) Project Work and Product Estimates	12
6.1) Estimate Summary	12
7) Project Resource Requirements	14
7.1) Staffing/Skill Requirements	14
7.2) Plan to Fill Skill Gaps	16
8) Dependencies and Constraints	17

8.1)	Constraints.....	17
9)	Risk Management	18
9.1)	Risk Management Strategy	18
9.2)	Initial Risk List.....	18
10)	Project Configuration and Data Management	19
10.1)	Configuration Management.....	19
11)	Project Process.....	20
11.1)	Software Life Cycle Model	20
12)	Referenced Documents	21
13)	Glossary	22
14)	Change Record.....	23

1) INTRODUCTION

The Owl Eye Monitor project will develop a system used to emulate stress tests on the KSU servers. By simulating real-world stress scenarios to generate controlled system problems such as CPU spikes, memory overloads, log errors, and database transaction failures. As well as using a Grafana-based Monitoring & Alerting system to visualize system metrics using Prometheus, detecting anomalies, sending notifications, and executing auto-resolution strategies.

2) PROJECT OVERVIEW

2.1) SCOPE

2.1.1) Identification

The system identified in this document is the OwlEye Monitor Application with Troublemaker. The software integrates the “Troublemaker” application, which intentionally generates various stress events, with a Grafana-based dashboard for monitoring, alerting, and automated incident resolution. This document applies to the entire suite of software components and infrastructure configurations associated with this project.

2.1.2) System Overview

The OwlEye Monitor system is designed to ensure the continuous health of applications and infrastructure by leveraging modern DevOps practices. Its primary purpose is to simulate real-world server anomalies through the Troublemaker application, which generates events such as CPU and memory load spikes, directory overflows, log errors, and simulated MySQL database error transactions. The system is operated on a dedicated Linux server and is integrated with a Grafana-based monitoring GUI to showcase alerts, send notifications, and attempt auto-resolution of issues. Sponsored by KSU and developed as part of a practical learning initiative, the project brings together system administrators, developers, and students to implement and maintain robust monitoring practices.

2.1.3) Document Overview

This document provides a detailed plan for the project, outlining the system’s identification, operational scope, and technical requirements. It encompasses the overall project name, objectives, constraints, and client information, ensuring that every stakeholder—from developers and system administrators to academic supervisors—has a clear understanding of the project’s vision and deliverables. The plan also covers policies, system descriptions, and the standards to be followed, serving as a comprehensive guide for the design, implementation, and maintenance phases of the project.

2.2) CURRENT SYSTEM OR SITUATION

Currently, there is no system in place to create or simulate high stress activities. This is the purpose of creating the troublemaker application.

For the monitoring system, there is a current system in place, but it is not as comprehensive as the monitoring system being developed. The current system does monitor the server’s health and performance in real-time, but it does not automatically send out emails when attacks are detected and does not attempt to perform an automatic resolution when the attack type has been identified.

2.3) BACKGROUND, OBJECTIVES, AND SCOPE

The current system exists as a simulated environment to provide practical experience in deploying modern monitoring and alerting systems. The mission is to establish a robust

framework that maintains system health by detecting and resolving anomalies in real time. This is achieved by integrating into the Troublemaker application, which generates controlled stress events, with a dashboard that monitors these events. The scope of the project encompasses the configuration of a dedicated virtual Linux server, the implementation of scripts for monitoring and alerting, and the simulation of various operational anomalies.

2.4) OPERATIONAL POLICIES AND CONSTRAINTS

Operational policies governing this project emphasize adherence to strict security practices, including access control, data integrity, and system availability. All simulated events and monitoring processes must comply with predefined safety and change management protocols to prevent unintended impacts on the virtual environment. The system is subject to constraints such as limited server resources, fixed simulation intervals, and integration boundaries with external components like the MySQL database and Grafana interface.

2.5) DESCRIPTION OF CURRENT SYSTEM OR SITUATION

The current system operates in a dual-mode environment: a regular monitoring state and a simulated stress-test state. Under normal operations, the system continuously monitors standard performance metrics and logs via the dashboard. During scheduled simulated events, the Troublemaker application intentionally induces high CPU and memory loads, as well as triggers error transactions in a MySQL database. These states allow the system to test its resilience and response mechanisms. The operational environment is a dedicated virtual server that interconnects with multiple subsystems, including automated scripts, log file monitors, and a real-time alerting framework.

2.6) USERS OR INVOLVED PERSONNEL

The system is intended for a diverse group of users, including DevOps practitioners, system administrators, software developers, and students engaged in hands-on learning. Key personnel include project managers, developers responsible for the Troublemaker application and Grafana dashboard, and operations staff who monitor system health and respond to alerts. These individuals work collaboratively, utilizing their specialized skills in system monitoring, incident response, and automation to ensure smooth operations.

2.7) STANDARDS TO FOLLOW

This project will follow IEEE Std 730-2014, IEEE Standard for Software Quality Assurance Plans, which provides a rigorous framework for planning, implementing, and monitoring quality assurance processes. This standard is important because it ensures that every phase of the project – from development through testing and deployment – meets stringent quality benchmarks. Additionally, the project will utilize IEEE Std 828™-2012, IEEE Standard for Software Configuration Management Plans, to systematically manage configuration items, control changes, and maintain consistency across all software artifacts. The application of these standards supports robust development practices, enhances traceability, and reinforces the reliability and maintainability of the monitoring system.

3) DEVELOPMENT BACKGROUND/APPROACH

The system will be developed for Linux operating systems and will consist of two major components. The first component is a troublemaker application, which will stress test the server by generating scheduled stress events. The second component being developed is a Monitoring/Alerting System which will be developed utilizing the open-source tools Grafana and Prometheus.

This system will consist of two primary development environments. The first being the traditional IDE which will be used to develop the system. The other being a server designated for testing and simulating troublemaker events.

3.1) HIGH LEVEL ESTIMATES

3.1.1) Project Plan and Design Phase

- **Total Effort:** 6 Weeks (1/6 – 2/14)
- **Lines of Code:** N/A
- **Pages of Requirements:** 23
- **Development Documentation:**
 - Project Plan
- **Test Cases:** N/A

3.1.2) Troublemaker Development Phase

- **Total Effort:** 5 Weeks (2/17 – 3/21)
- **Lines of Code:** TBA
- **Pages of Requirements:** TBA
- **Development Documentation:**
- **Test Cases:**

3.1.3) Monitor/Alert System Development Phase

- **Total Effort:** 4 Weeks (3/24 – 4/18)
- **Lines of Code:** TBA
- **Pages of Requirements:** TBA
- **Development Documentation:**
- **Test Cases:**

3.2) KEY CONTACTS AND STAKEHOLDERS

	Role:	Role Description:	Contact Info
--	-------	-------------------	--------------

Stakeholder Name:			Email:	Phone:
Taylor Cuffie			tcuffie@kennesaw.edu	470-578-5126
Alla Kemelmakher				470-578-5759
Rupesh Shirke			rupesh.shirke@outlook.com	
			rmith87@gmail.com	
			chaitanyapalanki@gmail.com	

4) FEATURES, PRIMARY DELIVERABLES, AND EXTERNAL COMMITMENTS

4.1) FEATURE LIST

4.1.1) Troublemaker

The troublemaker application will be used to generate a multitude of scheduled stress events on the server. Users of the troublemaker can create events and modify them to be able to be set when events are scheduled, choose what method to stress the system, and choose one of the varying levels of intensity.

- Generate Stress Events
- Configure Settings
 - Event Type
 - File Error Generation
 - Simulated Error in DB
 - Event Schedule
 - Time-Based
 - Choose Day(s)
 - Choose Time
 - Intensity
 - High
 - Medium
 - Low

4.1.2) Monitor and Alert System

The “Owl Eye” system is responsible for monitoring the server in real-time and detecting stress events generated by the troublemaker application. Additionally, when high stress events are detected, the system should send alerts via email notifying recipients of the occurrence.

- Real-time Server Monitoring
 - Display Metrics
 - Server Health
 - Performance
- Alert System
 - Send auto generated emails
- Resolution System
 - Deploy an auto-resolution based on determined attack type

4.2) CUSTOMER DELIVERABLES

1.) Project Kickoff Meeting:

- **Delivery Date:** 2025-01-24
- **Contents:**
 - Introduction to sponsors
 - Review of project goals, timeline, key contacts, and initial Project Scope

2.) Project Planning and Design:

- **Delivery Date:** 2025-02-16
- **Contents:**
 - Project Plan and Requirements documentation
 - Initial Design of the Troublemaker application and Monitoring system

3.) Project Design Document:

- **Delivery Date:** 2025-02-23
- **Contents:**
 - Software Design Specification Documentation

4.) Functional Troublemaker Application:

- **Delivery Date:** 2025-03-21
- **Contents:**
 - A working application capable of generating scheduled stress events on the server, including CPU spikes, memory overloads, directory overflows, log errors, and database transaction failures. Includes configuration options for event scheduling and intensity settings.

5.) Grafana-Based Monitoring and Alert System:

- **Delivery Date:** 2025-04-18
- **Contents:**
 - A real-time dashboard to visualize system metrics and monitor stress events.
 - Integrated alerting system to notify users via email or other messaging platforms.
 - Auto-resolution mechanisms to address common server issues automatically.

6.) Project Test Document:

- **Delivery Date:** 2025-04-20
- **Contents:**
 - Documentation containing test strategy, test cases, and results

7.) User Guide and Documentation:

- **Delivery Date:** 2025-05-05
- **Contents:**
 - A comprehensive user guide detailing how to operate, maintain and troubleshoot the system.
 - Technical Design Documents for future developers to use to understand and maintain the system

8.) Final Report and Presentation:

- **Delivery Date:** 2025-05-05

- **Contents:**
 - A project summary report detailing objectives, implementation, challenges, and future recommendations.
 - A recorded demonstration video showcasing the system's functionality.
 - GitHub Repository with the latest version of source code

5) PROJECT SCHEDULE

5.1) PROJECT MILESTONES

Date (YYYY-MM-DD)	Milestone/ task	Deliverable	Remarks
2025-02-09	Project Plan Due (Product Inception)	Complete or In-Progress Project Plan document.	
2025-02-14	Project Planning and Design	Project plan and requirements document. Initial design of the Troublemaker application and monitoring system. Detailed use-case scenarios for testing.	
2025-03-21	Troublemaker Application Development	Functional Troublemaker application capable of generating stress events. Adding configuration options for scheduling and intensity.	
2025-04-18	Monitoring and Alerting System Development	Complete and Functioning Monitoring System. Grafana-based monitoring dashboard.	

5.2) PROJECT STATUS TRACKING & WORKING MEETING MINUTES

Meeting # - Date	Attendees	Description	Duration
Meeting 1 – 01/22/2025	Ian Riggins, Isa Siddique, William Walker, Huy Nguyen, Evan Valencia	Prep for meeting with sponsors. Discussed what questions we should ask the sponsors about the project and decided on a team background	1 hour
Meeting 2 – 01/24/2025	Ian Riggins, Isa Siddique, William Walker, Huy Nguyen, Evan Valencia	Kick off meeting with sponsors, introduced ourselves as a team and got to meet the sponsors	45 minutes
Meeting 3 – 2/31/2025	Ian Riggins, Isa Siddique, William Walker	Weekly meeting with sponsors where we	30 minutes

		learned that we need to do the SCP form 2/Documentation before anything else	
Meeting 4 – 2/2/25	Evan Valencia, Ian Riggins, Isa Siddique	Created the Project plan and started filling out sections and creating a schedule to finish the documentation before next team meeting.	1 Hour

6) PROJECT WORK AND PRODUCT ESTIMATES

This section provides a breakdown of the estimated effort, time allocation, and resources required to develop the **OwlEye Monitoring System** project. The estimates consider the work breakdown structure, major milestones, and required deliverables.

6.1) ESTIMATE SUMMARY

The development effort is divided into three key phases:

1. **Project Planning & Design**
2. **Troublemaker Development**
3. **Monitoring & Alert System Development**

The tables below summarize the product size and effort estimates:

Project	Estimate Attributes		
	Size		
WBS areas	Unit of Size	Size	Effort
Total Requirements Effort (includes feature-related <u>and</u> "other" (non-feature) Requirements work)			35
Feature Related Requirements Size and Effort Totals	PAGES	21	~35
Total Development Effort (includes feature-related <u>and</u> "other" (non-feature) Development work)			167
Feature Related Development Coding Size and Effort Totals	LOC	~500	142
Feature Related Development Documentation Size and Effort	PAGES	20-50	~25
Total Testing Effort (includes feature-related <u>and</u> "other" (non-feature) Testing work)			25
Feature Related Testing Size and Effort Totals	TEST CASES	20-30 test cases	25
Feature Level Effort Total (from Feature Estimate Worksheet)	Hours		35
Development Effort Total (Includes Feature Level and project level overhead for Requirements, Development, and Testing)	Hours		167
Project Level Effort Total (from Project Level Effort Estimates worksheet, excluding requirements, development, and testing)	Hours		80
Project Total Effort (Project Totals + Feature Totals)			282

The project work is broken down into individual tasks with estimated efforts.

6.1.1 Project Planning & Design (6 weeks: Jan 6 – Feb 14)

Tasks & Effort Estimates:

- Understand project requirements and deliverables – 3 hrs
- Develop the **SCP-Form02 Project Plan** – 10 hrs
- Design system architecture (server setup, data flow, event triggers) – 10 hrs
- Define milestones and tracking approach – 2 hrs
- Research and evaluate tools (Grafana, Prometheus, MySQL, Python) – 6 hrs
- Create Use-Case Scenarios for testing(system overview, architecture, team roles) – 4 hrs
- **Total Estimated Time: 35 hours**

6.1.2 Troublemaker Application Development (5 weeks: Feb 17 – Mar 21)

Tasks & Effort Estimates:

- Develop event simulation for CPU, Memory, Disk, MySQL, Log errors – 21 hrs
- Implement configuration options (event scheduling, intensity) – 15 hrs
- Write unit tests for event triggers & stress tests – 10 hrs
- Debug and optimize event generation – 8 hrs
- Create **Troublemaker User Guide** – 8 hrs
- **Total Estimated Time: 62 hours**

6.1.3 Monitoring & Alerting System Development (4 weeks: Mar 24 – Apr 18)

Tasks & Effort Estimates:

- Set up **Grafana & Prometheus** dashboard – 15 hrs
- Connect Prometheus to monitor server metrics (CPU, RAM, logs) – 20 hrs
- Implement email notification system for alerts – 10 hrs
- Develop **Auto-Resolution Scripts** (file cleanup, service restarts) – 20 hrs
- Write and execute test cases for monitoring & alerting system – 15 hrs
- Debug and optimize monitoring dashboard – 8 hrs
- **Total Estimated Time: 80 hours**

7) PROJECT RESOURCE REQUIREMENTS

7.1) STAFFING/SKILL REQUIREMENTS

Member: Evan Valencia

Role: Team Leader

Critical Skills:

- Technical Skills
 - o Python and Java
 - o Git
- Soft Skills
 - o Time Management

Skill Gaps:

- Shell
- Linux
- SQL

Member: Ian Riggins

Role: Development Manager

Critical Skills:

- Technical Skills
 - o Programming Languages
 - C#
 - Java
 - Python
 - o Source Control Management
 - Git
 - Repository Management
 - o SQL
 - o Unit Testing
- Project Management Skills
 - o Software System Design
 - o Software System Architecting
 - o Code Reviews
- Problem-Solving Skills
 - o Critical Thinking
 - o Troubleshooting

Skill Gaps:

- Shell Scripting
- Linux

- UI Design

Member: Isa Siddique

Role: Planning Manager

Critical Skills:

- Technical Skills
 - o Technical Documentation
 - o Java Knowledge
 - o C# Knowledge
- Soft Skills
 - o Attention to Detail
 - o Time Management
 - o Analytical Skills

Skill Gaps:

- Python
- Shell
- Linux

Member: William Walker

Role: Quality & Process Manager

Critical Skills:

- Technical Skills
 - o SQL
 - o Java
 - o Shell Script
 - o Unit Testing
- Soft Skills
 - o Communication
 - o Problem Solving

Skill Gaps:

- Python
- Linux

Member: Huy Nguyen

Role: Support Manager

Critical Skills:

- Technical Skill

- SQL
- C#
- Java
- HTML
- Soft Skills
 - Brainstorm
 - Documenting

Skill Gaps:

- Linux
- Python
- UI Design

7.2) PLAN TO FILL SKILL GAPS

Reading and Watching documentation on the shell, python, and Linux systems. Hands-on practice to apply learned concepts into practical exercises and setting up test environments to gain experience. Seeking feedback from group members, and participating in code reviews, and code exercises to track progress.

8) DEPENDENCIES AND CONSTRAINTS

8.1) CONSTRAINTS

- The System must run on a test server.
- The System must run specifically on Linux.

9) RISK MANAGEMENT

9.1) RISK MANAGEMENT STRATEGY

1.) Risk Identification

- **Goal:** Recognize all potential risks that could impact project's success
- **Actions:**
 - Conduct meetings with team members discussing potential risks

2.) Risk Assessment

- **Goal:** Prioritize risks based on impact and probability
- **Actions:**
 - Classify risks
 - Rate likelihood of risks occurring
 - Identify the impact and its severity

3.) Risk Mitigation

- **Goal:** Develop Strategies to avoid or reduce risk impact
- **Actions:**
 - Implement preventive measures for high likelihood risks
 - Develop contingency plans to handle risks

9.2) INITIAL RISK LIST

Risk number	Risk Priority (H, M, L)	Likelihood of Occurrence	Risk name: brief description	Mitigation Strategy "ACCEPTED" or "MITIGATED" with pointer to plan.
1	M	Somewhat Likely	The Test Servers and Software Not functioning properly or crashing.	Mitigated: The team will reach out to the sponsors and communicate any technical issues involved with the test servers and software.
2	L	Not Likely	Not completing the Project on time with all deliverables.	Mitigated: The team members will communicate with each other to work on the project and its deliverables.
3	L	Not Likely	Not meeting with the stakeholders to satisfy needs	Mitigated: The team members will communicate with each other to ensure that the stakeholder's needs of the project are completed and satisfied.

10) PROJECT CONFIGURATION AND DATA MANAGEMENT

10.1) CONFIGURATION MANAGEMENT

Configuration Items (CIs):

- **Stress Test Scripts:** Scripts for generating specific stress scenarios (CPU spikes, memory overload, etc.).
- **Configuration Files:** YAML/JSON files defining test parameters (load thresholds, failure modes).
- **Monitoring Dashboards:** Grafana templates for visualizing performance metrics.
- **Prometheus Rules:** Custom alerts and threshold configurations.
- **Notification Channels:** Configuration for alerts via email, Slack, or other messaging platforms.
- **Auto-Resolution Strategies:** Scripts to restart services or adjust resource allocation automatically.

Version Control

- **Repository Structure:** Maintain version-controlled repositories (e.g., Git) for all configuration items.
- **Branching Strategy:** Use feature, development, and release branches for isolated development and deployment.
- **Tagging and Release Management:** Tag stable configurations with semantic versioning.

11) PROJECT PROCESS

11.1) SOFTWARE LIFE CYCLE MODEL

This project is set to be complete in a span of approximately two and a half months. Due to limited time and in order to facilitate project completion, this project will adapt an Agile model for its Software Life Cycle Model specifically using FDD (Feature-driven development).

12) REFERENCED DOCUMENTS

No documents referenced

For now...

13) GLOSSARY

Term:	Definition:
FDD	FDD stands for Feature-driven development. It is an Agile model of software development that utilizes a series of techniques in order to meet its development goals. Techniques include creating feature lists, constructing an entire model, designing, developing, and building through features.
KSU	Kennesaw State University
SQL	SQL stands for Structured Query Language. It is a programming language that is used to manage and manipulate relational databases.

14) CHANGE RECORD

Issue #	Change Request	Reason for Changes	Description of Changes	Person	Date (YYYY-MM-DD)
1	01	Formatting	Adjusted the formatting of the plan template for easier readability.	Ian Riggins	2025-02-02
2	???	Initial workings of Project Plan	Started Project Introduction, Project Schedule, Project Resource Requirements, Project Constraints, & Glossary sections	Isa Siddique	
			Evan Valencia		
3	???	Updates to Project Plan	Started Development Background/Approach section	Ian Riggins	2025-02-04
			Started Project Process section		
			Updated Glossary		
4	???		Updated Resource Requirements with Strengths and Weaknesses	Huy Nguyen	2025-02-05
			William Walker		
5	???		Worked on the Risk Priority Section	Isa Siddique	
6	???		Worked on Section 4.2 and Project Work and Product Estimations Section	Evan Valencia	2025-02-06
7	???		Started Features Section	Ian Riggins	
8	???		Worked on the Project Overview Section	Huy Nguyen	
9	???		Worked on Project Management plan	William Walker	2025-02-07
10	???		Fixed Formatting Issues	Ian Riggins	
			Added Current System/Situation and High Level Risk Management Strategy		

10	???	Updates to Project Plan	Modified Customer Deliverables List	Ian Riggins	2025-02-07
----	-----	----------------------------	--	-------------	------------